

Software Engineering

Builds and Releases

Builds and Releases: Developer Platforms

- Track and organise versions of the code
- Execute automated actions (e.g. tests)
- Build and release software packages
- Using all of these features, we can build a **deployment pipeline**

Automated change management documentation:

Every change to the software application or system is documented by the deployment pipeline

Infrastructure as code helps to not only document software changes but also changes to hardware and configuration.

Database scripts that are checked in and executed automatically help to understand database changes



version control



release
management

Build and Release Automation

Build and Release Automation

- In Build and Dependency Automation we compile our software
 - starting the **compiler** and handing over
 - the **path** to work on
 - the **source files** to compile
 - the **dependencies** – local or from a repository – for those files
 - tons of **parameters**.
- In release management
 - we strive to produce and deliver valuable software in short cycles
 - and ensure that the software can be released reliably at any time

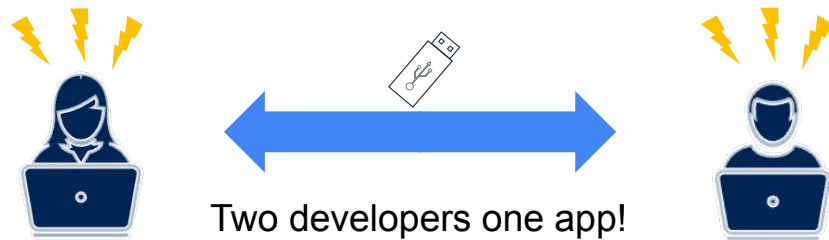
Best automate all the things that are repetitive.

In a way that it works the same locally (on your computer)
and remotely (on the build server)

Collaborating on code: Merging

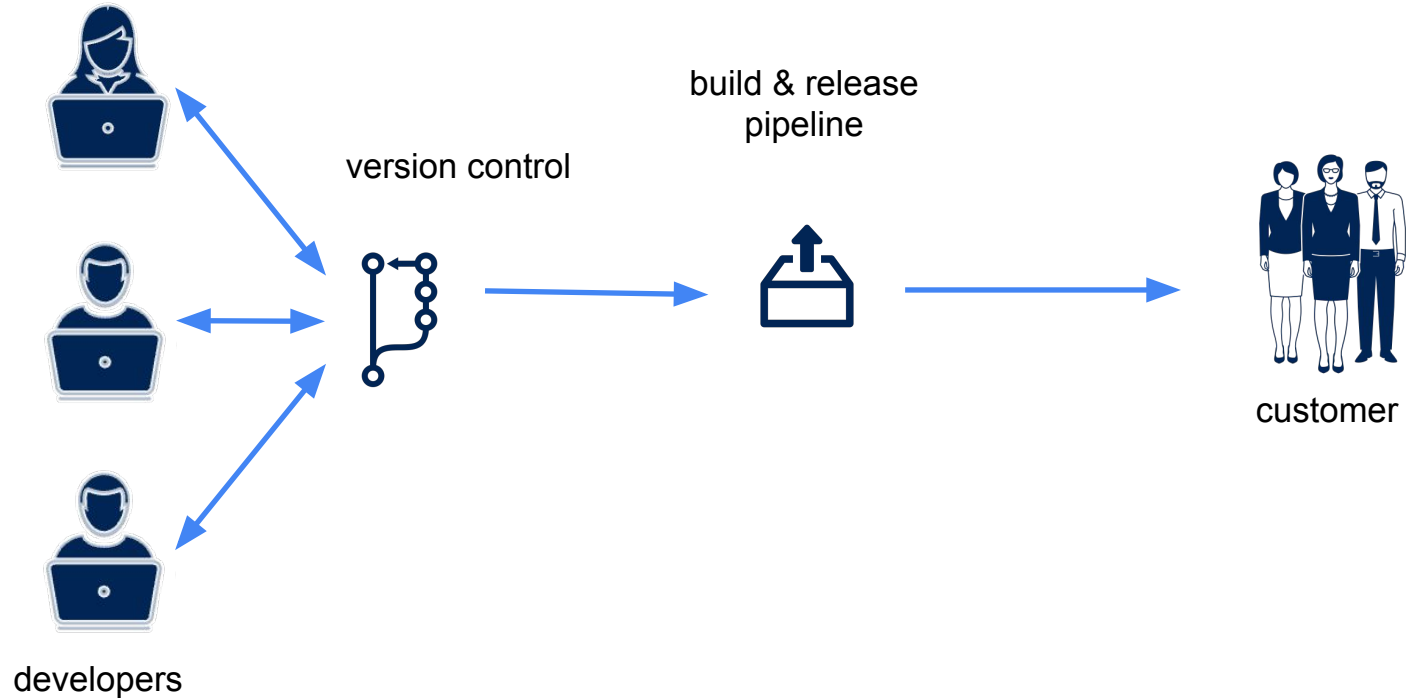
Collaborating on code: file exchange

- How do two or more developers work together?
- How do they share code?
 - Sandra and Peter work together on creating one app
 - Both of them sat together, came up with a **initial file structure** and **shared local copies**
 - Then they begin to **work in parallel** on the **same file on their own local copies**
 - Peter sends Sandra the new version of this file
 - Sandra needs to **check for differences and update her own file accordingly**
 - Peter get's the **new version of the file** from Sandra, has to **check for differences** again and **update his file**

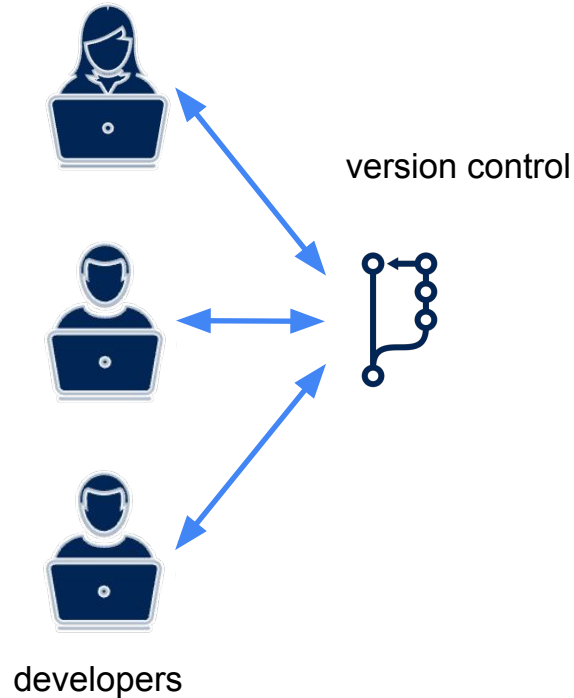


This is the worst possible way to people could work together on the same code base!
Error prone and hard to maintain. Not scalable.

Developer Platforms



Collaborating on code: version control systems



- Sandra and Peter work together on creating one app
- Both of them sat together, came up with a initial file structure and uploaded it into the version control system
- Then they begin to work in parallel on the same file on their own local copies
- Peter uploads new version of this file into the version control system
- Sandra retrieves the file from the version control system and updates her own file accordingly, supported by the system
- Sandra uploads her version into the system
- Peter get's the new version of the file from the system
- Marc retrieves the file as well

Collaborating on code: Merging

Merging is the process of integrating code changes into a code base.

When using version control systems, we eliminate the need to merge manually across every developer's computer and put a mediator in between.

One-way merge means that just one source changed: either the local code (on the developer's computer) or the remote code (on the version control system).

These are easily automated and need no manual effort.

Two-way merge means that just two source changed: both the local code and the remote code. Some of these can be automated. Some require manual effort.

When performing merges, it's important to to check for **merge errors**. These include:

Syntax errors: code that is not executable - checked automatically by Compile and Type Checks

Semantic errors: code that makes no sense to execute - best checked by Static Code Analysis and Automated Testing - if the test coverage is sufficient. Otherwise hard to detect.

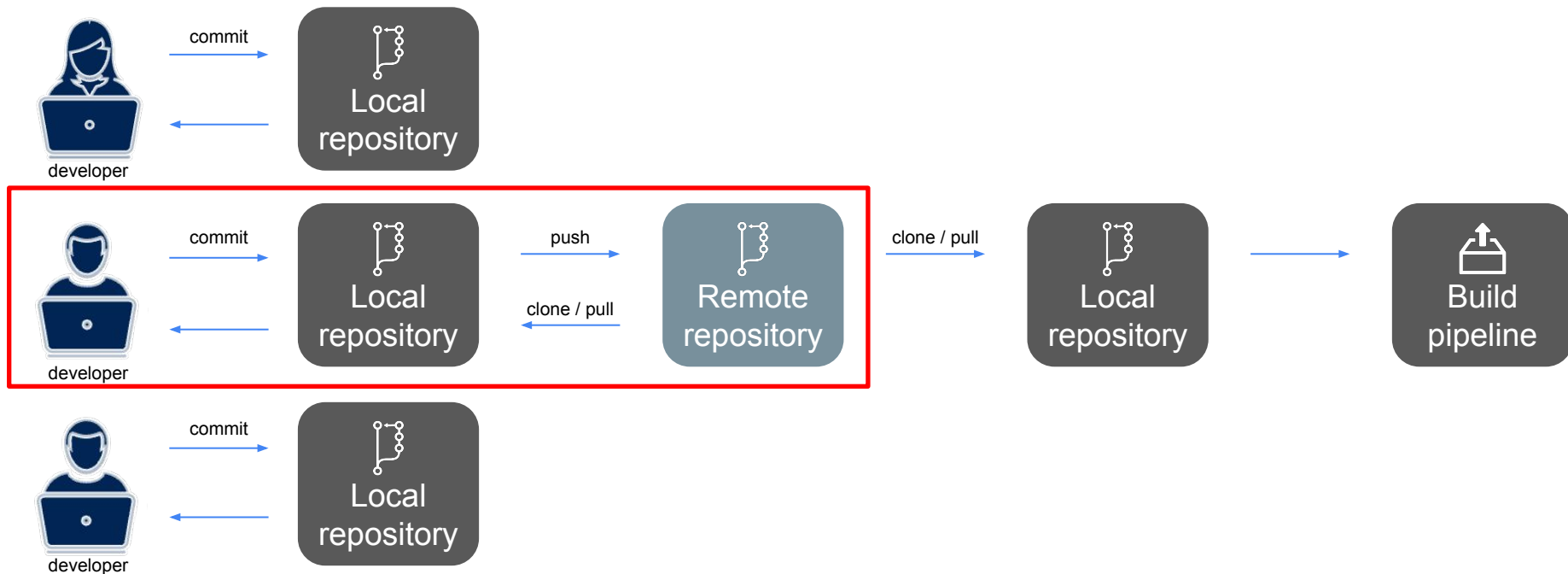


Version Control System

- Simplifies collaboration on shared items
- Establishes change tracking on source files through **commit history**
- Allows members to restore a previous state
- Typical Cloud developer platforms that include version control:
 - [GitHub.com](https://github.com)
 - [GitLab.com](https://gitlab.com)

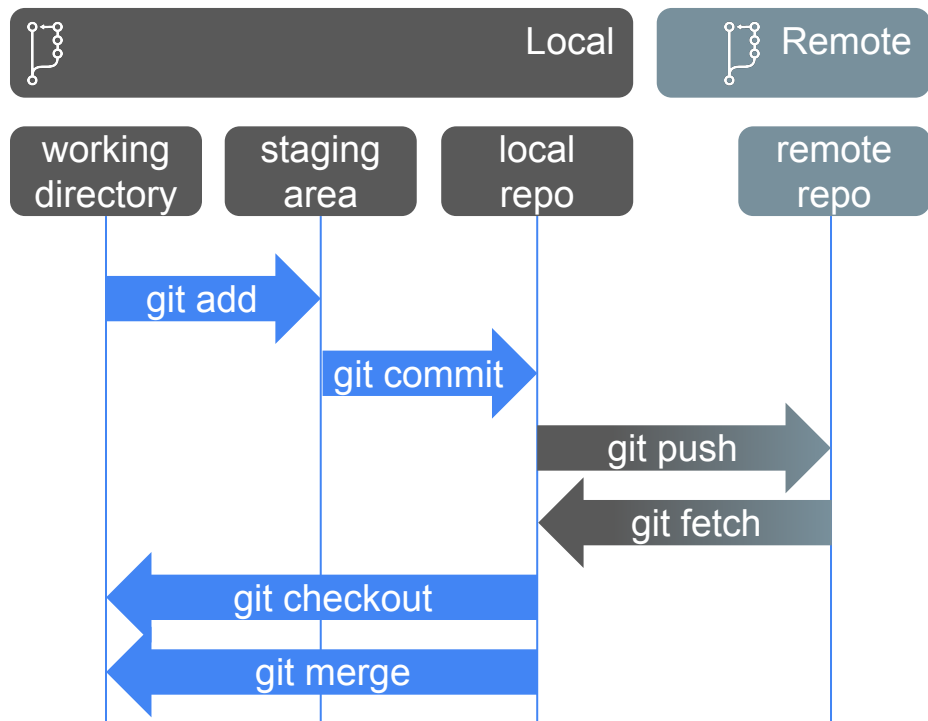
Distributed Version Control Overview

Developers and other systems connect to a remote via a local repository



Overview of Git Commands

- **git add**
 - Add changes from working copy to staging
- **git commit**
 - Saves selected changes of staging to local
- **git push**
 - Uploads local commits to remote
 - Can cause conflicts if remote has changed
- **git pull (fetch & merge)**
 - Downloads and merges from remote
 - Can cause conflicts when same line has changed
- **git clone (fetch & checkout)**
 - Copies a complete repository from remote to new local working copy



Live Coding: Demonstration of merging (errors)

- Help me find the speed that is helpful to you
 - Ask questions
 - Make suggestions
 - Say if I should slow down or repeat

Good practises for working with version control

- Commits
 - Rather have **more commits (smaller)** than large ones with lots of changes
 - Make **logically linked changes together** in one commit
 - Run **build before committing** so that commits always function properly
 - **Push only when unit test** have run without failure
 - **Push** as **often** as possible
- Keep your local updated
- Commit messages should follow rules
 - Should say what this commit does (*adds PartnerIntegration class, fixes crash on startup, etc*)
 - Should contain name or user ID and ticket number ([UID123][US-2001])

Collaborating on code: Versioning

Version Management

Version	A specific state of a software application or library
Release	A formally approved (tested, validated, documented and accepted) version of a software
Commit Hash	A unique identifier to identify the version of the software internally in the version control system
Tag	A pointer to a specific commit hash in the version control system, see git documentation

Semantic Versioning (see: [SemVer.org](#)) - Example: 2.1.23

A unique identifier within the scope of a specific software to identify the version **externally**

Given a version number MAJOR.MINOR.PATCH, increment the:

MAJOR version when you make incompatible API changes

MINOR version when you add functionality in a backward compatible manner

PATCH version when you make backward compatible bug fixes

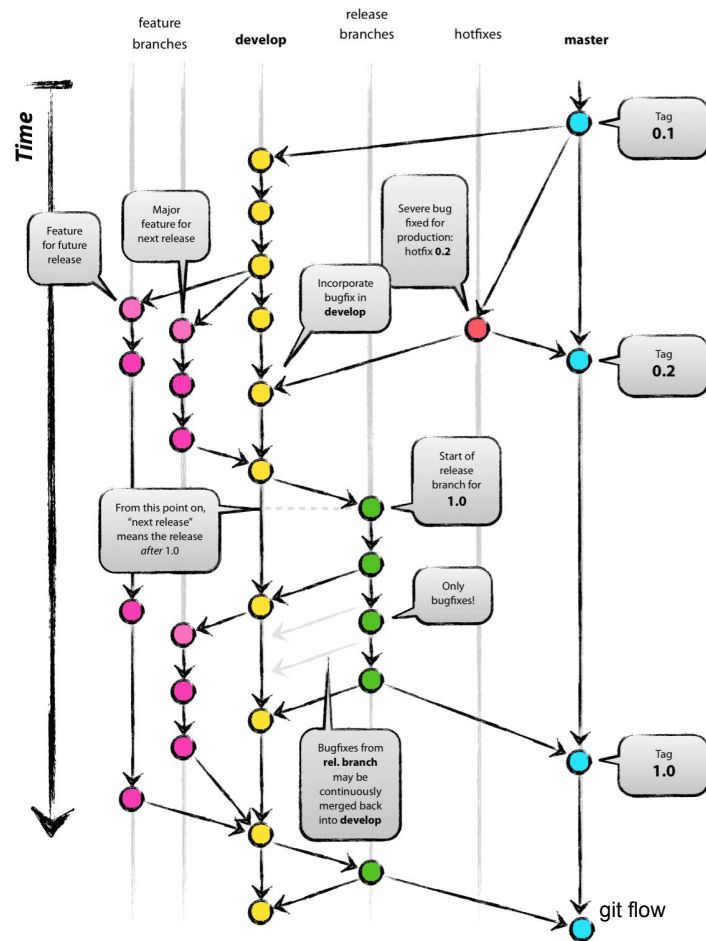
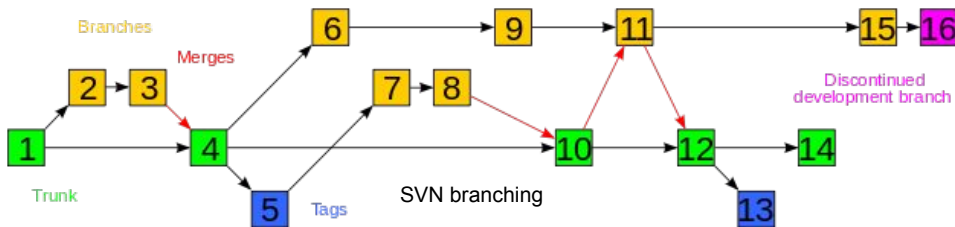
Collaborating on code: Branching

Branching

- **Motivation: only integrate tested code**
- Branching
 - Split off one timeline (dev, feature, release, fix, master, etc.)
 - Enables parallelization of work at the expense of integrating (aka merging) later
 - Enables multiple versions to be maintained

Be careful: excessive branching makes merging harder

Long living branches mean big bang merges that more likely result in merge errors



Branching

- Git-Flow

- feature branch => a specific feature / user story
- develop branch => main develop timeline
- hotfix branch => timeline for fixes
- production branch => timeline for releases
- master branch => main timeline

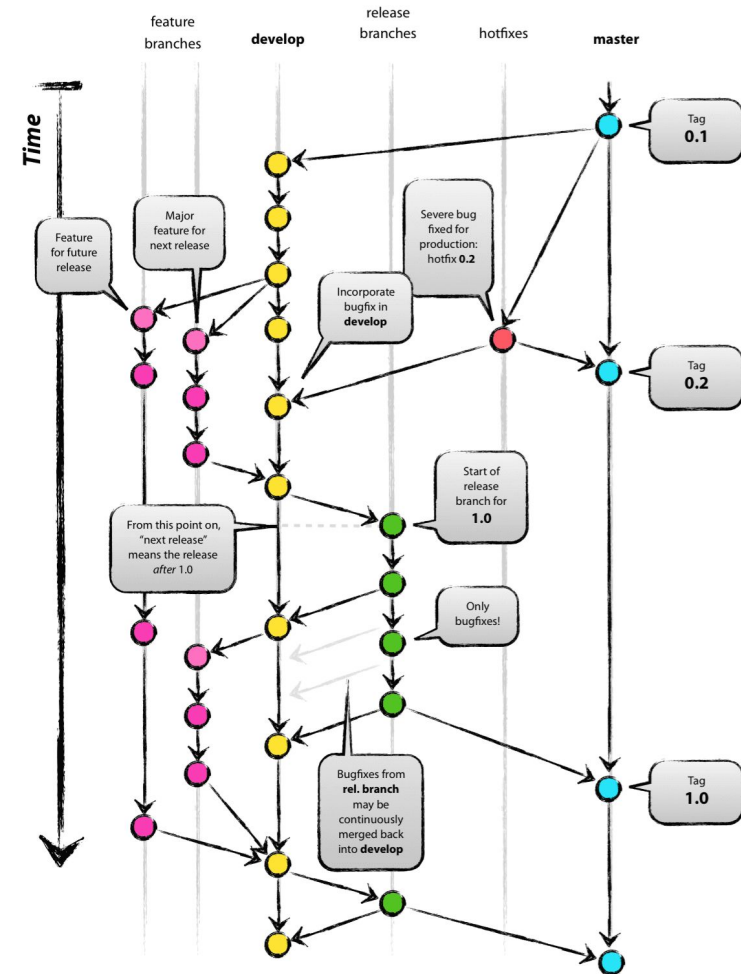
Advantages:

- Standardised branching concept

Disadvantages:

- Complicated, thus making human error likely

Git Flow is helpful if you need a high amount of MANUAL regression testing - if not, it's overhead



Recap: Software Testing: Basic Definitions

Manual Testing: Testing software by humans executing test cases through a User Interface to identify defects or issues.

Automated Testing: Using scripts and tools to automatically execute test cases and compare actual outcomes with expected results.

Regression Testing: Re-executing previous test cases on updated software to ensure that new changes haven't introduced new defects in existing features.

Important: Manual Regression Testing is time intense and therefore increases personnel costs and time to market

Collaborating on code: CI/CD

Definitions for CI and CD

- **Continuous Integration (CI)**

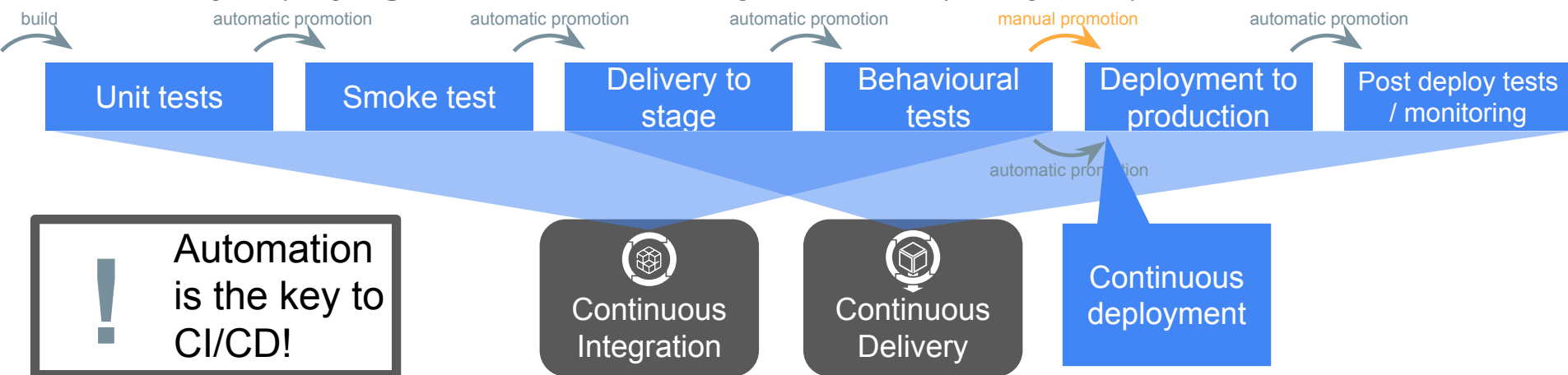
Integrating (merging) code **into the main branch** regularly (multiple times per day)

- **Continuous Delivery (CD)**

Making sure that the **main branch** is always in a **releasable** state (by test automation)

- **Continuous Deployment**

Actually deploying the main branch to **production** (live system) once it's releasable



How to reduce need for branching and enabling CI/CD?

Motivation for branching:

only integrate tested code

Feature Toggles enable us to integrate (merge) **untested code without executing it, avoiding big bang merge conflicts** and allowing us to **switch features on and off** in a running system

before

```
function reticulateSplines() {  
    // current implementation lives here  
}
```

Code source: [M. Fowler Blog](#)

after

```
function reticulateSplines() {  
    var useNewAlgorithm = false;  
    // UNCOMMENT IF YOU ARE WORKING ON THE NEW SR ALGORITHM  
    // useNewAlgorithm = true;  
  
    if( useNewAlgorithm ){  
        return enhancedSplineReticulation();  
    }else{  
        return oldFashionedSplineReticulation();  
    }  
}  
  
function oldFashionedSplineReticulation() {  
    // current implementation lives here  
}  
  
function enhancedSplineReticulation() {  
    // TODO: implement better SR algorithm  
}
```

Branches and Feature Toggles

They are not mutually exclusive.

You can use both and migrate from one technique to the other.

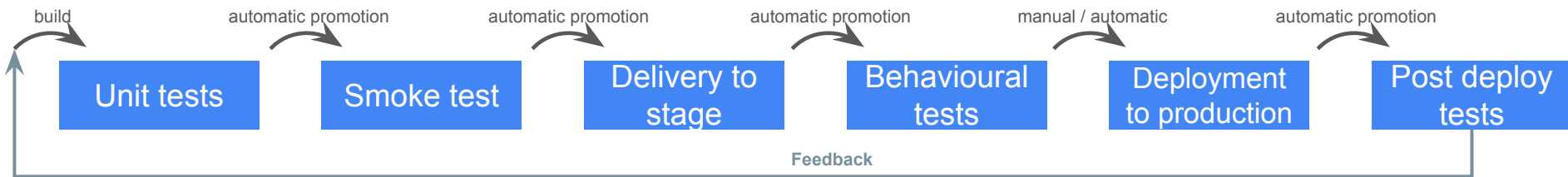
Just always make sure you never execute untested code on production.

Using none of these techniques and just deploying untested code is not an option.

(Antipatterns: “Commit and run”, “Banana Software”)

Continuous Delivery / Deployment

- Clear benefits
 - Accelerated **time to market**: faster release cycles and therefor market advantage
 - Improved **quality** and customer satisfaction: more feedback gathering
 - Improved **efficiency**: automation!
 - Reduced **risks**: vacations, errors in manual release steps
- But also a lot of challenges
 - High level of professional skills required



Measuring Software Engineering Performance

Measuring Software Engineering Performance

How would you measure Software Engineering Performance?

Measuring Software Engineering Performance

Never: on the **individual** level.

Why not?

Measuring Software Engineering Performance

Never: on the **individual** level.

Individual performance goals feed **ego** trips and reduce
collaboration

Measuring Software Engineering Performance

Measure the **system**, not the individuals.

Why?

Measuring Software Engineering Performance

Measure the **system**, not the individuals.

This enables them to inspect and adapt
without blaming each other.

Measuring Software Engineering Performance

The 4 key metrics by the DORA team of Google

Deployment Frequency

Lead Time for Changes

Change Failure Rate

Time to Restore Service

Source: [DORA \(DevOps Research and Assessment\)](#)

Measuring Software Engineering Performance

The 4 key metrics by the DORA team of Google

Deployment Frequency—How often an organization successfully releases to production

Lead Time for Changes—The amount of time it takes a commit to get into production

Change Failure Rate—The percentage of deployments causing a failure in production

Time to Restore Service—How long it takes an organization to recover from a failure in production

Source: [DORA \(DevOps Research and Assessment\)](#)

